

COMPENG 2DX3
Final Project Report

Hasan Asim
April 5, 2025

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by **[Hasan Asim, asimh6, 400512182]**

Device Overview

Features

Integrated 3D Scannar System

- Motor/ToF Module for distance measurements
- Captures orthogonal displacement samples
- Covers 360 degrees
- VL53L1X ToF Sensor
 - Maximum measuring range: up to 4 meters
 - Ranging accuracy: ± 20 mm
 - Operating voltage: 2.6V – 3.5V
 - \$8 - \$15 CAD
- ULN2003 Stepper Motor
 - 512 steps per full rotation
 - Operating voltage: 5V – 12V
 - \$5 - \$10 CAD

MSP432E401Y Microcontroller

- 20 MHz Bus Speed
- Powered by an ARM Cortex-M4F processor
- Integrated status LEDs (PF0, PF4, PN1):
 - Measurement transmission indicator
 - UART Tx indicator
 - Return-to-home indicator
- Onboard buttons (PJ0, PJ1) for manual control
- Serial Communication via COM3
- Baud rate: 115200 BPS
- Flash Memory: 1 MB
- SRAM: 256 KB
- Around 75\$ CAD

Data Communication

- I2C protocol for communication between MSP microcontroller and ToF sensor
- UART protocol for serial communication between the microcontroller and a PC

Data Visualization

- Python visualization script
- Generates 3D point cloud models
- Exports data as .xyz files for mapped spatial visualization

General Description

The device is an integrated device designed to perform 3D spatial mapping through distance measurements captured across multiple 360° orthogonal sweeps. It constructs a three-dimensional representation of the scanned environment by systematically collecting and processing spatial data.

The system is built around three key components: a VL53L1X Time-of-Flight (ToF) sensor, a ULN2003-controlled stepper motor, and an MSP432E401Y microcontroller. The microcontroller governs the system's operation while coordinating motor steps, managing communication with the sensor, and sending collected data to a computer via UART serial connection.

The stepper motor rotates the ToF sensor in fixed angular increments (e.g., 11.25°), allowing it to scan a full vertical plane. As it turns, the sensor emits infrared light and calculates the distance to nearby objects based on the time taken for the reflected signal to return. Distance readings, accurate up to 4 meters within ± 20 mm, are transferred via an I²C interface to the microcontroller for forwarding.

Collected data is sent to a computer, where Python scripts process and visualize the readings as a 3D point cloud. This enables the creation of a virtual model of the physical space, showcasing the scanner's ability to map environments efficiently and accurately.

Block Diagram

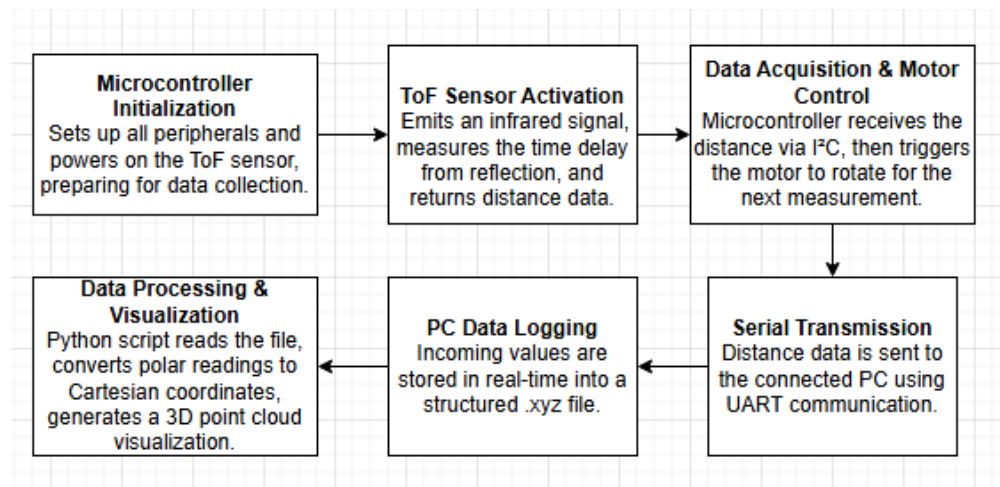


Figure 1: Data flow graph

Device Characteristics Table

MSP432E401Y		ULN2003		VL53L1X	
Bus Speed	20MHz	+	5V	V _{in}	3.3V
Baud Rate	115200	-	GND	GND	GND
Measurement Status	PF0	IN1	PH0	SDA	PB3
UART Tx	PF4	IN2	PH1	SCL	PB2
Returned Home Status	PN1	IN3	PH2		
Serial Port	COM3	IN4	PH3		
Start	PJ1				
Stop	PJ0				

Figure 2: Device Characteristics Table

Device Description

Distance Measurement

The VL53L1X Time-of-Flight sensor is used to measure distances by emitting infrared light pulses. These pulses reflect off objects and return to the sensor, where the time taken for this is measured. By knowing the speed of light, the sensor can calculate the distance to an object using the formula: $D = \frac{(\text{Speed of light})(\text{Time taken})}{2}$. The division by two accounts for the fact that the photon travels to the object and then back to the sensor. Once the distance is calculated, the sensor converts this measurement into a digital signal, which is then transmitted to the microcontroller through the I²C communication protocol.

Before the sensor can be used for measurements, several hardware and software components need to be set up. The microcontroller's system clock is configured with the PLL_Init() function, and a delay timer is initialized via SysTick_Init() to allow for timed operations. Port configurations are also crucial: Port B is set up for I²C communication with the sensor, Port G controls the XSHUT pin for hardware resets, Port H manages the stepper motor, and Port J enables user input via onboard buttons PJ1/PJ0. The I²C_Init() function configures I²C0 on Port B, adjusting settings for the master clock, open-drain for the SDA line, and the appropriate

communication speed (100 kbps). This setup allows the microcontroller to function as a I²C slave device as it sends commands to the VL53L1X.

To reset the ToF sensor, the microcontroller uses the VL53L1X_XSHUT() function, which drives the XSHUT pin low, disabling the sensor, and then allows it to enter a high-impedance state so it can begin the boot process. The microcontroller repeatedly checks the sensor's boot status using VL53L1X_BootState() until the sensor signals that it is fully initialized. Next the sensor is fully initialized through the VL53L1X_SensorInit() function, which loads the sensor's default settings and prepares it for normal operation. Once initialized, the sensor begins to take measurements using the function VL53L1X_StartRanging(dev), which triggers the sensor to start emitting laser pulses and timing the reflection of these pulses from objects in its path. The sensor is mounted on a rotating motor shaft, allowing it to scan in a 360° radius. The motor is programmed to rotate the sensor incrementally by 11.25° per step, meaning the motor completes 32 steps to make a full rotation. Each step results in a distinct distance reading from a specific direction, effectively turning the sensor into a basic LiDAR system capable of scanning the surrounding environment. To ensure proper synchronization, after every 11.25° increment of the motor, the system prompts the ToF sensor to take a new distance measurement. This ensures that each reading corresponds to a specific angle, forming a coherent scan of the environment. The microcontroller waits for the measurement to be completed by polling the sensor with the function VL53L1X_CheckForDataReady(). Once the data is ready, it is retrieved using VL53L1X_GetDistance() and then transmitted to a PC via UART for further processing and visualization.

Visualization

To facilitate the visualization of the rotating LiDAR system's output, two Python scripts were developed: transfer_data.py and visualize.py. These scripts work right after another to collect raw distance data from the VL53L1X time-of-flight sensor, convert it into 3D Cartesian coordinates, and ultimately render a mesh-like 3D point cloud using Open3D.

The process begins with transfer_data.py, which establishes a serial communication channel with the microcontroller through COM3 at a baud rate of 115200. The script then resets both the input and output buffers to ensure a clean communication stream. After the user input specifies the number of full scans to be taken, the script enters a loop where it continuously reads distance data line-by-line via UART. Each received value is decoded from UTF-8 and cleaned of carriage return characters. The script then calculates the corresponding angle of measurement based on the number of microsteps that the stepper motor has taken. This angle is then used to convert the polar distance measurement into Cartesian coordinates: the y-coordinate is computed as $y = r \cdot \cos(\theta)$, the z-coordinate is computed as $z = r \cdot \sin(\theta)$, and the x-coordinate starts at 0 and increments by a constant value (100 mm in this case) after each full 360° scan.

This approach simulates translational motion along the x-axis, effectively creating a series of yz-plane cross-sections that build up into a 3D structure as scanning progresses. Each point's (x, y, z) values are written to a .xyz file named point_array.xyz. This format is simple and efficient for storing 3D data.

The second script, visualize.py, handles the rendering of this data. It uses the Open3D library to read the .xyz file into a point cloud data structure. The script first prints the numerical values of the points for verification before opening an interactive visualization window that displays the raw point cloud. To enhance spatial interpretation, the script assigns a unique index to each point and constructs a set of lines connecting them. These lines are created in two stages. First, points within the same scan are connected sequentially. Secondly, corresponding points between consecutive scans are connected, forming a 3D grid-like structure that helps visualize how the environment changes across space.

The final output is a wireframe-style 3D model that shows the shape of the scanned area using connected slices of points. Open3D's LineSet makes it easy to render and explore the model interactively. Together, the two scripts handle everything from collecting the data to visualizing it. It also supports modular expansion, filtering out bad data, or even turning it into a full surface. It's a solid way to turn microcontroller readings into something you can actually see and work with.

Application Note

The 3D spatial mapping system, using a Time-of-Flight (ToF) sensor mounted on a rotating motor, has great potential for interactive displays or exhibits, especially in places like museums or science centers. As the ToF sensor continuously scans its environment in 360-degree sweeps (at intervals like 11.25° per step), it captures detailed distance measurements within a 4-meter range. This data is then used to build a real-time, dynamic 3D map of the space.

The distance measurements are converted from polar to Cartesian coordinates, creating a 3D point cloud that tracks movement and interaction within the exhibit. For example, as visitors move through the exhibit area, the system can adjust the visuals or trigger interactive elements based on their position. This makes the experience much more engaging, allowing the environment to respond to the visitor's actions in real time.

The system also uses range status data to indicate the accuracy of each measurement, ensuring that only valid, reliable data is used to control the display. This prevents errors and ensures the system only responds to meaningful movements, offering a smooth and responsive experience.

By integrating this 3D mapping system into exhibits, museums and science centers can create more interactive and immersive environments that encourage exploration and enhance learning.

Whether for educational exhibits or art installations, the system brings digital content to life in ways that traditional static displays can't, offering a truly dynamic experience for visitors.

Instructions

This guide assumes that the user has already installed the necessary software, including the Kiel Development Environment, an appropriate IDE for running the Python script, and a compatible version of Python with Open3D.

1. **Connect the Microcontroller:** Connect the microcontroller to your computer and identify the correct COM port. To do this, press the Windows key, search for 'Device Manager,' and open it. In the Device Manager window, expand the 'Ports (COM & LPT)' section, then locate the COM port number next to 'XDS110 Class Application/User UART (COM #)'.
2. **Modify the Python Script:** Open the `transfer_data.py` script and go to line 7. Change the line `s = serial.Serial('COMX',115200)` to the correct COM port number from step 1.
3. **Set Up the Circuit:** Follow the wiring provided in the Circuit Schematic below to correctly set up your circuit.
4. **Configure the Kiel Project:** Open the Kiel project, navigate to 'Options for Target,' and select the 'C/C++' tab. Set the optimization level to '-O0'. Then, under the 'Debug' tab, choose 'CMSIS-DAP Debugger' from the dropdown list and click 'Settings.' In the settings window, select 'XDS110 with CMSIS-DAP' in the top-left dropdown menu. Click 'OK' to apply the changes.
5. **Adjust Python Script for Desired Increments:** In the `transfer_data.py` script, go to line 18 and change the value of increment to the desired distance in millimeters between scans (e.g., 100mm).
6. **Install Missing Dependencies:** Ensure all required libraries are installed. If any dependencies are missing (such as `serial`, `math`, `numpy`, or `open3d`), open a terminal and use `pip install`.
7. **Build and Load the Project:** In the Keil project, click 'Translate,' followed by 'Build,' and then 'Load' to compile the project and upload it to the board.
8. **Press the RESET button** to flash the project onto the microcontroller board.

9. Run the Python Script: Execute the transfer_data.py script to begin the process.
10. Confirm Data Reception: Enter how many scans you want to take.
11. Complete all Scans: Press PJ1 to initiate the 360-degree rotation scan. After every scan, move the device in the x direction by the set increment value(mm) and press PJ1 again. Repeat until all scans are done.
12. Visualize: Once all the data is scanned, execute the visualize.py to visualize all the data points using Open3D.

Expected Output

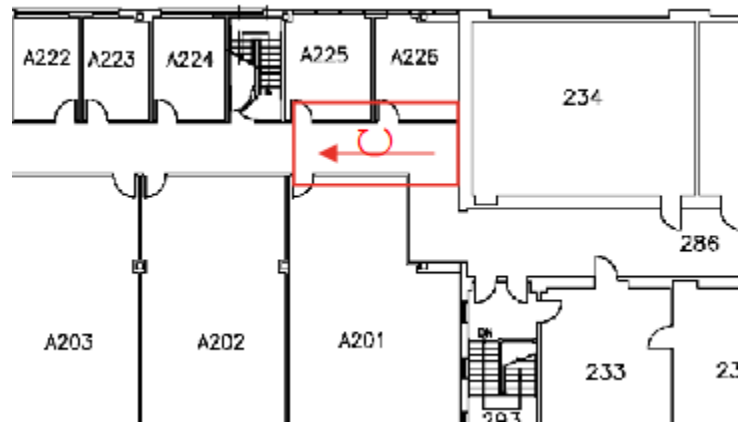


Figure 3: Hallway location

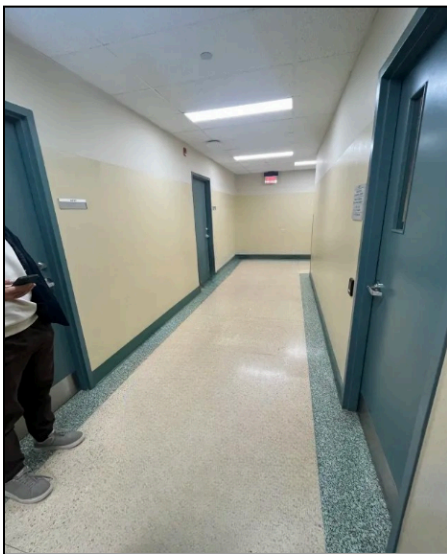


Figure 4: Hallway location

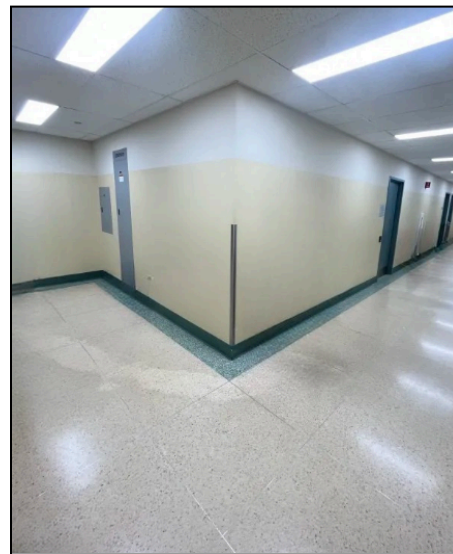


Figure 5: Hallway location

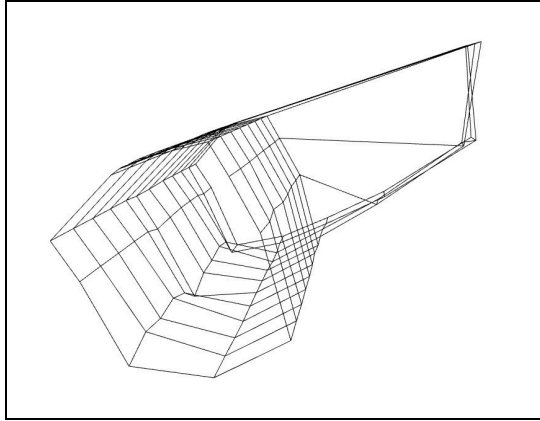


Figure 6: 3D Visualization

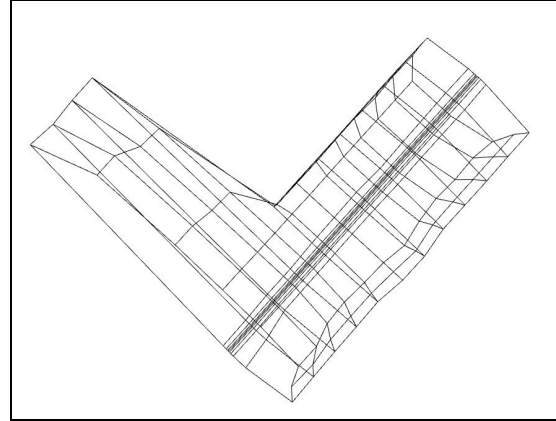


Figure 7: Top View of 3D Visualization

Limitations

1. The ARM Cortex-M4 processor comes equipped with a 32-bit Floating Point Unit (FPU), which supports precise floating-point operations such as addition, subtraction, multiplication, and division. Since coordinate transformations rely heavily on trigonometric calculations, maintaining high precision is important. For this reason, computational tasks like these are better handled by the PC, which can perform 64-bit floating point math and has significantly higher processing power, ensuring both improved accuracy and faster performance.
2. The quantization error of a system using an ADC can be estimated with the formula:

$$(\text{Maximum measurable distance}) \div (2^{\text{number of ADC bits}})$$
 For our Time-of-Flight sensor, the maximum measurable distance is 4000 mm. Assuming a 12-bit ADC, the expected quantization error is about 0.97 mm, which aligns closely with the device's stated resolution of 1 mm.
3. By checking the Device Manager on Windows, under the “Ports (COM & LPT)” section, and looking into the properties of a communication port, the listed maximum transmission rate is 128000 bits per second. In this project, the UART communication was configured to use a baud rate of 115200 bps, which ensures compatibility while staying within the allowable range.
4. Communication between the microcontroller and the ToF sensor is established using the I²C protocol. In this setup, the microcontroller operates as the master, while the ToF module functions as the slave. Although the I²C bus was configured to run at 100 kbps (standard mode), the microcontroller supports high-speed communication up to 3.33 Mbps, and the VL53L1X sensor is capable of handling up to 400 kbps (fast mode),

according to its specifications.

5. One of the main factors limiting the system's speed is the stepper motor. To prevent skipped steps or motion failure, a delay of at least 2 milliseconds is needed between each phase. Testing showed that a full 360° rotation introduces a delay of approximately 1.024 seconds due to the motor's stepping requirements (4 phases per step × 512 steps × 2 ms per phase). Another timing bottleneck is the sensor itself, which requires around 200 milliseconds per reading during rotation, based on delay measurements taken during operation.
6. The system clock speed was configured to 20 MHz by adjusting the PLL settings. Specifically by using the following parameters: PSYSDIV = 3, Q = 0, N = 4, MINT = 16, MFRAC = 0, the system clock frequency is computed as:

$$\begin{aligned} & ((25,000,000 / ((0 + 1) * (4 + 1))) * (16 + (0 / 1024))) / (3 + 1) \\ &= 80,000,000 / 4 \\ &= 20 \text{ MHz} \end{aligned}$$

Circuit Schematic

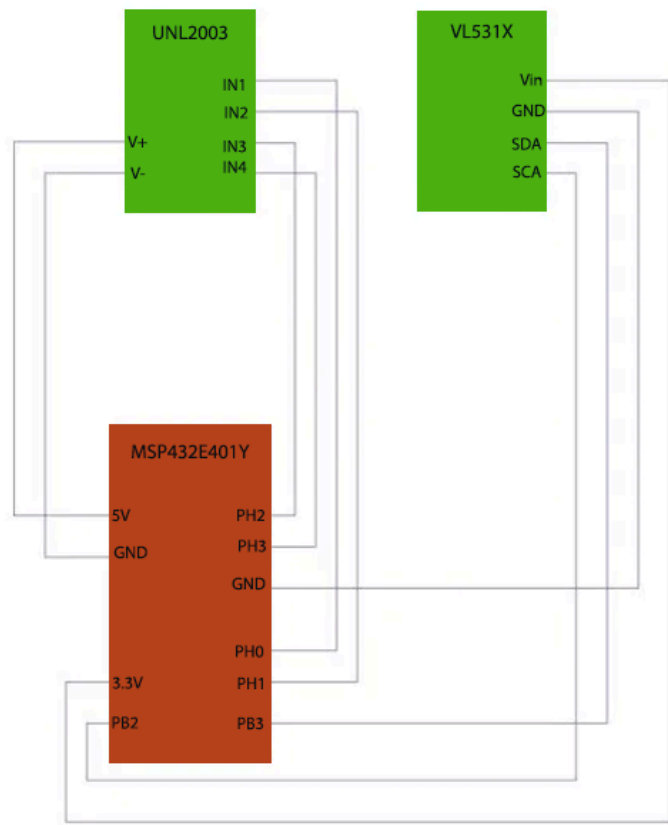


Figure 8: Circuit Schematic

Programming Logic Flowcharts

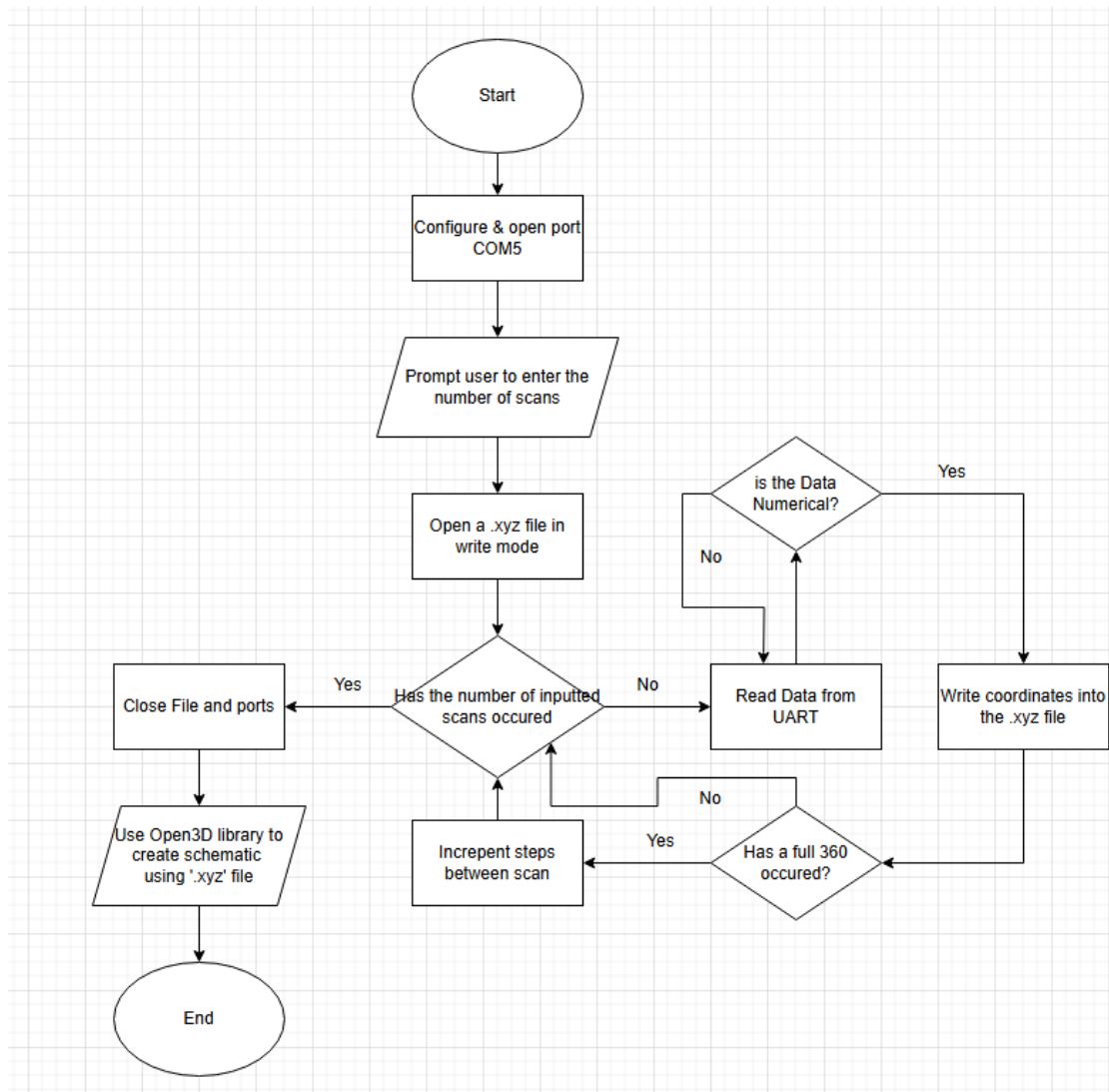


Figure 9: Keil Program Flowchart

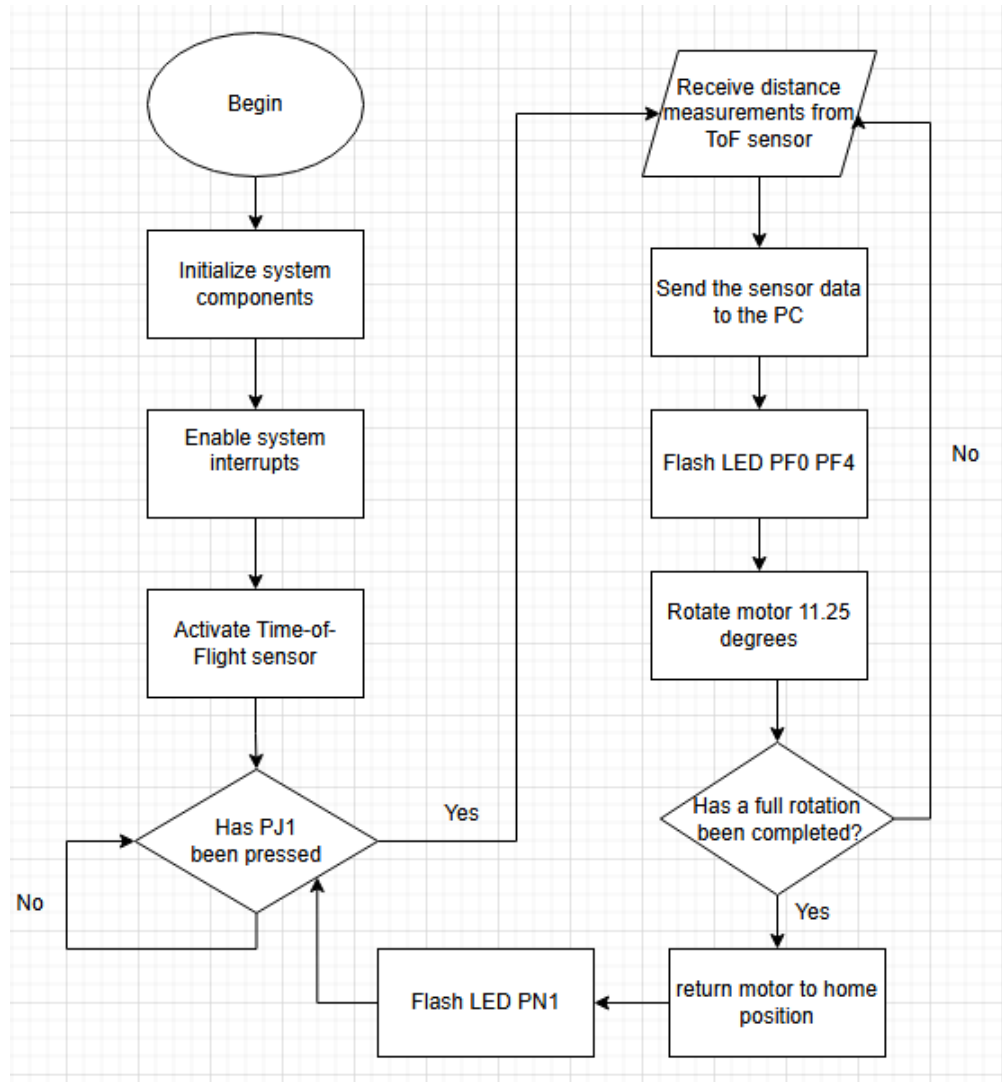


Figure 10: Python Program Flowchart

References

- [1] “MSP432E401Y SimpleLink™ Ethernet Microcontroller Data Sheet,” Ti,
<https://www.ti.com/lit/ds/symlink/msp432e401y.pdf> (accessed Apr. 6, 2025).
- [2] “VL53L1X,” STMicroelectronics,
<https://www.st.com/en/imaging-and-photonics-solutions/vl53l1x.html> (accessed Apr. 6, 2025).